

Pipeline de Datos

Proyecto Car Seats

ETL · EDA · Dashboard Interactivo · Modelos Supervisados

Programación para la Ciencia de Datos

Docente: Mariela Moraga

Estudiante: Milenka Guerra

Junio 2026

Objetivo del Proyecto

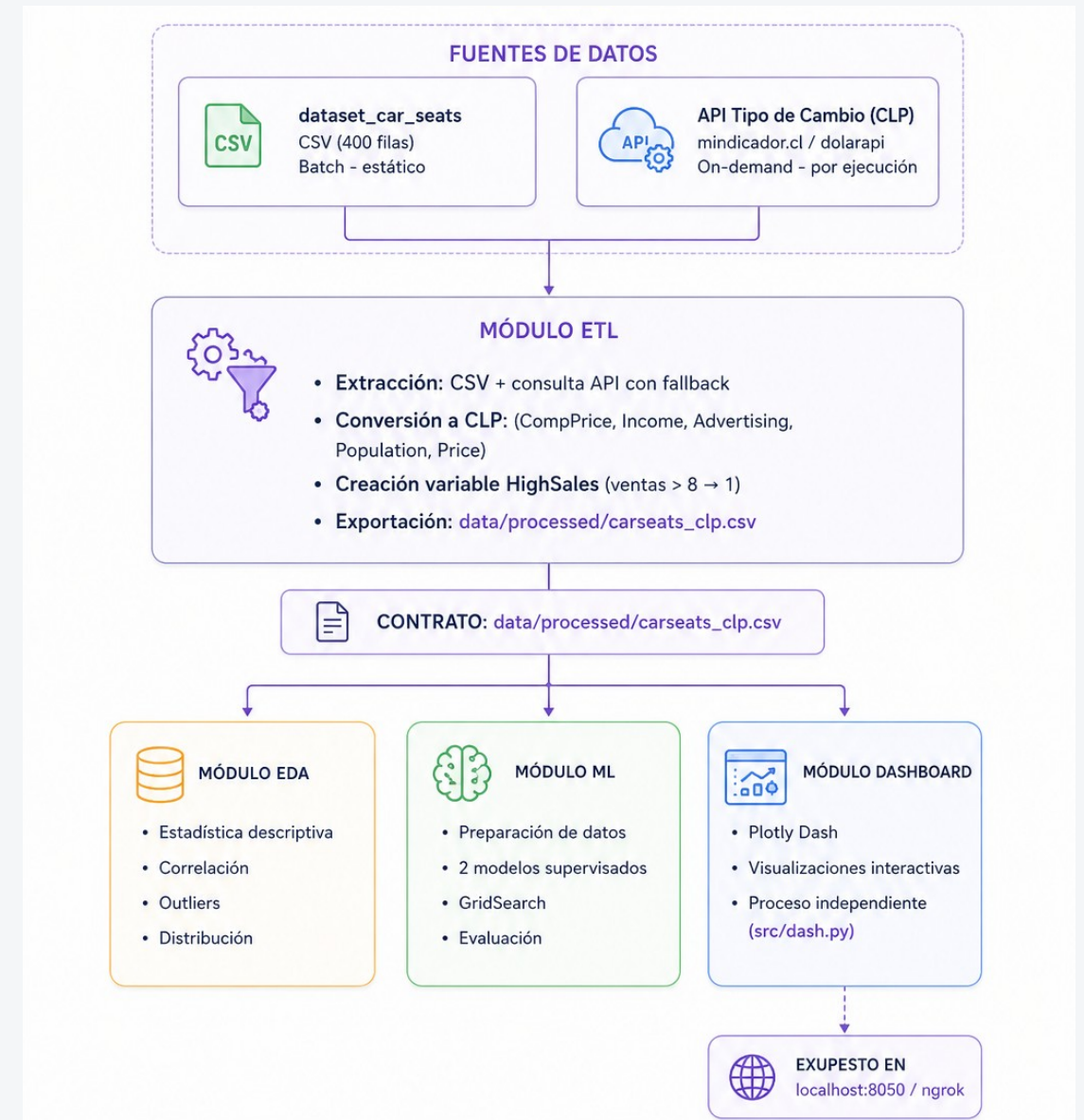
- Construir una solución de análisis de ventas para el dataset Car Seats.
- Integrar 2 fuentes de datos: dataset CSV (400 tiendas) + API REST de tipo de cambio.
- Predecir si una tienda tendrá ventas altas o bajas (HighSales) con modelos supervisados.
- Comunicar los resultados con un dashboard interactivo diferenciado por audiencia.

Arquitectura Técnica

Pipeline Híbrido Modular

Se descartaron Lambda y Lakehouse: no hay tiempo real ni volumen de datos que las justifique. → Es sobreingeniería para este proyecto.

- Cada módulo (ETL, EDA, Modelado, Dashboard) es independiente.
- Se comunican solo a través de `data/processed/carseats_clp.csv`.
- El dashboard vive en proceso aparte (`src/dashboard.py`): no bloquea la ejecución del notebook.



ETL: Fuentes y Transformaciones

- Fuente 1: dataset_car_seats.csv (400 tiendas, 11 columnas).
- Fuente 2: API de tipo de cambio, con fallback en cadena (dolarapi.com → mindicador.cl → valor fijo).
- Transformación: 4 columnas monetarias convertidas de USD a CLP.
- Creación de HighSales (Sales > 8): 59% bajas / 41% altas.
- Manejo robusto de errores: try/except en carga de datos y en cada fuente de la API.

Tiendas
400

Columnas Finales
12

Ventas Altas
41%

Nulos/Duplicados
0

EDA: Principales Hallazgos

- ShelfLoc es la variable más asociada a las ventas: 15% de ventas altas en tiendas Bad vs 78% en tiendas Good.
- Price (-0.44) y Advertising (+0.27) son los predictores numéricos más fuertes, con sentido económico claro.
- CompPrice tiene correlación débil con Sales (0.06) de forma aislada, pero resultó relevante en el modelo multivariado.
- Dataset bien comportado: distribuciones casi simétricas, muy pocos outliers (máximo 5 de 400 casos, en Price).

Dashboard: Demo en Vivo

- Vista Ejecutiva: 4 KPIs + 3 gráficos con conclusión de negocio (insight callouts), sin filtros.
- Vista Analítica: 1 dropdown (ShelveLoc/Urban/US) que mueve 3 gráficos enlazados.

- Valor de negocio:
 - ¿Conviene ser más barato que la competencia? (56% vs 16% de ventas altas).
 - ¿Vale la pena invertir en publicidad? (61% vs 31% de ventas altas).

Modelos Supervisados: Comparación

- 2 modelos: Regresión Logística y Random Forest.
- Ambos optimizados con GridSearchCV.
- Ganadora: Regresión Logística, en CV, en test y en la curva ROC.
- Variables más influyentes en ambos modelos: Price, ShelveLoc_Good, Advertising → confirman el EDA.

Recall en clase 'Altas': 0.91 (Reg. Logística) vs 0.79 (Random Forest) → detecta más tiendas de venta alta sin sacrificar precision.

F1 (test) — Reg. Logística
0.90

F1 (test) — Random Forest
0.83

ROC-AUC — Reg. Logística
0.98

ROC-AUC — Random Forest
0.94

Git y GitHub

- Una rama por etapa: feature/etl, feature/eda, feature/dashboard, feature/modelos, feature/docker, docs.
- Cada rama converge en develop antes de integrarse a main.
- Commits descriptivos por hito ('Etapa ETL finalizada', 'Etapa EDA finalizada'...).
- Repositorio remoto en GitHub, con ramas efectivamente pusheadas.

```
$ git log --oneline --graph --all --decorate

* 847e6da (develop, feature/modelos) Desarrollo de
  modelo de Regresión Lineal y Random Forest
* af82015 (feature/dashboard) Dashboards y
  conexión con ngrok establecida
* e2dd4e4 (feature/eda) Etapa EDA finalizada
  (Sección 2)
* 0971ef7 (feature/etl) Etapa ETL finalizada
  * 04c6c5e (main) Estructura inicial ev3 SCY1101
```

Lecciones Aprendidas y Mejoras Posibles

Técnicas

- Separar procesos de larga duración (dashboard) del notebook evita bloquear la reproducibilidad.
- El modelado multivariado reveló relaciones (CompPrice) invisibles en el EDA univariado.

Mejoras Futuras

- Ajustar el umbral de clasificación según costo de negocio real.
- Agregar la interacción Price-CompPrice como feature explícita.
- Docker: investigado, no implementado en esta entrega por tiempo.

Proyecto Car Seats
¡Gracias!